

A LESSON IN RESEARCH METHODOLOGY

Performance Estimation and Evaluation

Metrics, Measurement, and Scaling

Where Unit II closes its loop: the models of Lecture 5 meet a stopwatch, under the evidential standards of Lecture 2 and the empirical discipline of Lecture 4.

Research Methodology in CS · Unit II · Lecture 6



Performance Is Not a Number

“Faster” is not a property of a system. It is a property of a system, on a workload, on a platform, under a metric, summarised by a statistic — and changing any one of the five can reverse the answer.

- 1 This session closes Unit II's loop. Lecture 5 built models that predict performance; Lecture 4 set the standards for empirical claims; Lecture 2 said what admissible evidence looks like. Performance evaluation is where all three meet a stopwatch.
- 2 Most disputed performance claims are not dishonest. They fail on a summary statistic, an unrepresentative workload, or a baseline nobody tuned — each of which is a methodological choice, made silently and rarely reported.
- 3 The failure modes are so consistent that the field has codified them. Jain's 1991 text, the SPEC and TPC run rules, and the statistically-rigorous-evaluation literature all exist because the same mistakes kept recurring.
- 4 The discipline it demands is unglamorous: control what you can, report what you cannot, and state precisely which of the five dependencies your number is conditional on.

WHAT EVERY NUMBER DEPENDS ON

System · Workload · Platform
Metric · Summary statistic

A performance claim that names fewer than five of these is under-specified. It cannot be reproduced by anyone else, and it cannot be contested on its merits — only believed or disbelieved.

Write all five into the sentence that states your result.

Three Techniques for a Performance Answer

Jain's classification. They differ in when they can be applied, what they cost, and how much a reader will believe them.

TECHNIQUE 1

Analytical

Solve a model — Lecture 5's method.

AVAILABLE

At any stage, including before the system exists.

ACCURACY

Low to moderate; entirely dependent on the assumptions.

COST AND TIME

Small. Hours to days, and no hardware.

BEST FOR

Trends, limits, and “what happens at ten times this load”.

TECHNIQUE 2

Simulation

Execute a model of the system.

AVAILABLE

At any stage, including before the system exists.

ACCURACY

Moderate; limited by model detail and by run length.

COST AND TIME

Medium. Days to weeks, plus statistical analysis.

BEST FOR

Realistic structure that admits no closed form.

TECHNIQUE 3

Measurement

Run the real system and time it.

AVAILABLE

Only once the system exists, in a runnable configuration.

ACCURACY

High if done carefully, and worthless if not.

COST AND TIME

High. Hardware, isolation, and many repetitions.

BEST FOR

The configuration you actually have — and nothing beyond it.

Jain's rule: use at least two, and reconcile them. Agreement is far more persuasive than either alone — and disagreement is usually where the finding is.

A Systematic Procedure

Jain's ten steps, grouped into five phases. Each phase exists to prevent one specific, well-documented mistake.

PHASE	WHAT IT INVOLVES	THE MISTAKE IT PREVENTS
Frame	State the goals of the study and define the system boundary — what is inside the thing being evaluated and what is environment.	Evaluating a component while stating a conclusion about the whole system.
Specify	List the services the system offers and their possible outcomes; then select the metrics that capture them.	Choosing the metric after seeing which one flatters your system.
Parameterise	List system and workload parameters, then select the subset you will actually vary as factors.	Varying whatever was easy to vary rather than what the outcome depends on.
Design	Choose the evaluation technique, select the workload, and design the set of experiments to run.	One-factor-at-a-time designs that cannot detect interactions.
Execute and report	Run, analyse, interpret — then present the data in a form a reader can check.	Reporting a mean with no dispersion, no baseline detail, and no statement of scope.

Notice that eight of the ten steps happen before any measurement is taken. That ordering is not bureaucratic: it is what makes the result falsifiable, exactly as in Lecture 2's five-stage empirical anatomy.

PART 1 → PART 2

From the procedure to the two choices that decide the answer

Before a single measurement is taken, two decisions have already fixed what the study can conclude: which metric, and which workload.

Next: metrics, summary statistics, the tail, and workload characterisation.

Choosing the Right Metric

Four families. The first two answer different questions and are routinely traded against each other, which is why reporting only one is rarely honest.

METRIC 1

Response time

Time from request to completion.
Smaller is better.

WHAT TO WATCH

What a user experiences. Must be reported as a distribution: a mean response time describes almost nobody's actual experience.

METRIC 2

Throughput

Work completed per unit time. Larger is better.

WHAT TO WATCH

What an operator cares about. Meaningless unless you state the offered load at which it was measured — throughput at saturation is a different claim.

METRIC 3

Utilisation

Fraction of time a resource is busy.
Neither better nor worse.

WHAT TO WATCH

A diagnostic, not an objective. A device at 100% utilisation is a bottleneck, not an achievement — and Lecture 5's laws let you compute it.

METRIC 4

Cost and error

Energy per operation, cost per transaction, failure and deadline-miss rates.

WHAT TO WATCH

Increasingly the metrics that decide real deployments, and the ones academic evaluations most often leave out entirely.

Batching improves throughput and worsens tail latency. Whenever a mechanism trades one metric for another, report both — otherwise the improvement you show is being paid for off-screen.

Which Mean Are You Taking?

The summary statistic is a methodological choice, not a formatting one. Choosing the wrong one can name the losing system as the winner — as the next slide demonstrates with numbers.

STATISTIC	WHAT IT COMPUTES	USE IT FOR	USING IT WRONGLY GIVES
Arithmetic	The sum of the values, divided by how many there are.	Quantities that add up: total times, total work, total energy — anywhere the sum is itself meaningful.	The wrong winner, whenever it is applied to rates or ratios.
Harmonic	The count divided by the sum of the reciprocals.	Rates measured over a fixed amount of work — MIPS, FLOPS, operations per second per benchmark.	Nothing: this is the correct choice for rates, and it equals total work over total time.
Geometric	The nth root of the product of the values.	Normalised ratios, such as speed-ups against a baseline. Its result does not depend on which baseline was chosen.	A number with no physical dimension — which is why SPEC uses it only for normalised ratios.
Median	The middle value of the sorted observations.	Skewed distributions, where a heavy tail drags the mean away from typical behaviour.	(Not an error — a different question. Always report it with a spread.)

The rule that catches most of it: if the quantity is a rate, never take its arithmetic mean. Average the times arithmetically, or the rates harmonically — they give the same answer.

Worked Example: The Wrong Mean Picks the Wrong Winner

Question: “Two machines run the same two benchmarks. Which machine is faster?”

THE MEASUREMENTS

	Bench 1	Bench 2	
work	100 M	100 M	ops

Machine A	1 s	100 s	
Machine B	10 s	10 s	

THE TWO SUMMARIES

rates (Mops/s)	B1	B2
Machine A	100	1
Machine B	10	10
arithmetic mean	A = 50.5	B = 10.0
harmonic mean	A = 1.98	B = 10.0

1

The tempting summary

Convert each benchmark to a rate, then average the two rates arithmetically. Machine A scores 50.5 Mops/s against B's 10.0, and appears five times better.

2

The reality check

Total work is 200 million operations. Machine A takes 101 seconds to do it; Machine B takes 20. B completes the same work in a fifth of the time — so the arithmetic mean picked the loser.

3

The correct summary

The harmonic mean of the rates gives A = 1.98 and B = 10.0 Mops/s. That is exactly total work divided by total time. It agrees with reality because it is reality, rearranged.

4

Why it happens

Averaging rates arithmetically implicitly weights each benchmark by its own running time, so the fastest case dominates the summary — and a machine can “win on average” while losing on every job you run.

This is not a contrived case. It is why SPEC reports a geometric mean of normalised ratios rather than an arithmetic mean of rates, and why Jain devotes an entire chapter to choosing the summary statistic.

Percentiles and the Tail

Latency distributions in real systems are skewed and often heavy-tailed, which makes the mean a poor description of anybody's actual experience.

WHY THE MEAN MISLEADS

A mean of 20 ms is consistent with 95% of requests at 5 ms and 5% at 300 ms.

Those are entirely different systems to a user, and the mean cannot distinguish them. The heavier the tail, the further the mean sits from both the typical case and the worst one.

Reporting only a mean is a construct-validity failure: the statistic is not measuring the property being claimed.

AND IT COMPOUNDS AT SCALE

A request that fans out to N services waits for the slowest of the N.

At N = 100, an event at one service's 99th percentile is expected on most requests. The tail of a component becomes the median of the system — Dean and Barroso's “tail at scale”.

So in a distributed system, the p99 of a dependency is not an edge case. It is the common case.

Report the distribution

At minimum: median, 95th, 99th, and maximum — together with how many samples produced them.

Percentiles do not average

You cannot take the mean of per-machine p99 values and call it the system p99. Aggregate the raw observations.

State the load

Tail latency is far more load-sensitive than the mean. A percentile without an offered load is not interpretable.

Run long enough

A p99.9 estimated from 500 requests rests on roughly one observation. Compute the sample size you need.

Workload Characterisation

Four kinds of load, ordered roughly from most realistic to most controllable. You cannot have both.

KIND 1

Real workload

The actual production load, measured in place.

THE TRADE

Maximally representative, entirely unrepeatable, and usually unavailable to anyone else. Excellent for validation; useless for a controlled comparison.

KIND 2

Trace-driven

A recorded log of real requests, replayed against the system.

THE TRADE

Repeatable and realistic. But a trace captures one period and one deployment, and replaying it loses the feedback between system speed and user behaviour.

KIND 3

Benchmark suite

Standard programs with published run rules.

THE TRADE

SPEC CPU, TPC-C, YCSB, DaCapo. Comparable across papers and across years — and, per Lecture 3, steadily eroded by Goodhart's law.

KIND 4

Synthetic

Generated load with tunable parameters, or a microbenchmark isolating one operation.

THE TRADE

Ideal for explaining a mechanism and for sweeping a parameter. Dangerous as evidence about whole-system performance, which it systematically distorts.

Whichever you choose, state how the workload relates to the population you are claiming about. That single sentence is Lecture 4's external-validity argument, in performance clothing.

Experimental Design: Factors and Levels

With k factors at l levels each, a full sweep costs l^k runs. These are the standard ways of spending fewer without learning less than you need.

DESIGN	WHAT IT DOES	RUNS	WHEN TO USE IT
One factor at a time	Vary a single input; hold everything else fixed.	$k(l-1)+1$	Only when factors are known not to interact — which is rarely established.
Full factorial	Every combination of every level of every factor.	l^k	Few factors, few levels. Reveals every interaction, at a cost that explodes.
2^k factorial	Every factor at just two levels, low and high.	2^k	Screening: finding cheaply which of many factors actually matter.
Fractional factorial	A carefully chosen subset of the full design.	2^{k-p}	Many factors, accepting that high-order interactions become confounded.
Randomised replication	Repeat runs, in randomised order.	$\times r$	Always. Randomising order converts systematic drift into random error.

One-factor-at-a-time is the default in student work and the design that cannot detect interactions — which, in computer systems, is precisely where the interesting behaviour lives.

Instruments: What You Can Actually Measure

Four instruments, answering different questions. The first tells you how slow; the others tell you why.

INSTRUMENT 1

Wall-clock timers

The ground truth for end-to-end latency and throughput.

THE CATCH

Check the clock's resolution, and never time an interval shorter than the timer call's own overhead. Prefer a monotonic clock over wall time.

INSTRUMENT 2

Hardware counters

Cycles, instructions, cache misses, branch mispredictions — via perf or PAPI.

THE CATCH

These explain why something is slow. Beware counter multiplexing: request more events than the hardware has registers and each is sampled part-time.

INSTRUMENT 3

Sampling profilers

Periodically interrupt execution and record the call stack.

THE CATCH

Cheap, and attributes cost to code. Beware skid and safepoint bias: a sample lands where the interrupt was taken, not necessarily where the time went.

INSTRUMENT 4

Tracing

Record every event of interest, with timestamps and context.

THE CATCH

Exact attribution, paid for with overhead that perturbs the very thing being measured. The heavier the tracing, the less it describes the untraced system.

Every instrument perturbs what it measures. Report the overhead of your apparatus, or demonstrate that it is negligible relative to the effect you are claiming.

Sources of Measurement Error

Six effects that routinely produce swings larger than the speed-ups being reported. Control each, or state that you did not.

Warm-up and JIT

Managed runtimes compile, deoptimise and recompile. Steady state can take thousands of iterations, and “startup performance” and “steady-state performance” are two separate claims.

Frequency scaling

Turbo and DVFS mean identical code runs at different clock rates depending on thermal and power history. Pin the frequency, or report honestly that you did not.

Memory layout bias

Link order, environment size, and address-space randomisation shift code and data alignment. Mytkowicz et al. showed this alone can produce swings exceeding many published speed-ups.

Co-tenancy

Other processes, other containers, and noisy neighbours on the same cloud host. Report the isolation you actually had — shared hardware makes three significant figures fictional.

NUMA and affinity

Which core a thread lands on, and which memory node its data sits on, changes results substantially and non-deterministically unless threads are pinned.

Cache and predictor state

Repeated runs on the same input warm caches and branch predictors that a cold production system would not have. Measure the state you intend to claim about.

Reporting a Performance Result

Five elements. Together they are what makes a number reproducible, and therefore contestable.

1

The configuration

Hardware, OS, compiler and flags, runtime versions, dataset, and the number of repetitions — enough that a stranger could rebuild it.

2

The distribution

Median and spread, or percentiles with the sample count. Never a bare mean, and never a single run.

3

The baseline

What you compared against, which version, and exactly how you tuned it. An untuned baseline invalidates a comparison rather than merely weakening it.

4

The statistic

Which mean you took and why that one; which test you used and why it suits this data's actual distribution.

5

The scope

The workloads, load levels and platforms over which the claim holds — and a plain statement that outside them it does not.

THE TEST TO APPLY BEFORE SUBMITTING

Could a competent stranger, holding only your paper and your artifact, obtain a number close to yours? If any of the five elements is missing, the honest answer is no — and Lecture 4's badge terminology gives you the vocabulary to say so.

ON SIGNIFICANT FIGURES

Report no more precision than your measurement supports. Four significant figures from runs whose variance is five per cent is a claim about the formatting of your spreadsheet, not about the system.

Speedup, Efficiency, and Scalability

Three definitions that look elementary and are misreported constantly — almost always through an unstated choice of baseline.

DEFINITION 1

Speedup

$$S(p) = T(1) / T(p)$$

How much faster the parallel version is than the sequential one.

Everything depends on what $T(1)$ means: the best known sequential algorithm, or your own parallel code run on one core? Only the first is an honest baseline.

DEFINITION 2

Efficiency

$$E(p) = S(p) / p$$

The fraction of ideal speedup that p processors actually delivered.

Falling efficiency is the normal case, and it is the interesting quantity. Plotting only speedup lets a badly scaling system still look like a rising line.

DEFINITION 3

Strong vs weak scaling

fixed problem vs fixed work per p

Strong scaling holds the total problem fixed; weak scaling grows it with p .

They answer different questions and produce different curves. Saying only “it scales” without saying which is not a claim a reader can evaluate.

Speedup measured against a deliberately poor sequential implementation is the performance world's strawman baseline — Lecture 4's failure mode, with a rising graph attached to it.

Worked Example: What Amdahl's Law Forbids

Question: “Five per cent of this program is inherently serial. How much can parallel hardware possibly help?”

THE MODEL

f serial fraction of the work
 $1 - f$ perfectly parallelisable
 p number of processors

$$S(p) = 1 / (f + (1 - f)/p)$$

THE PREMISE DOING THE WORK

A fixed problem size, with a serial fraction that does not change as p grows. Everything the law forbids follows from that one assumption.

This is a Lecture 3 deduction carried out inside a model — and its premise is precisely what Gustafson's reframing attacks on the next slide.

1

Set up

Normalise the total sequential time to 1. The serial part costs f no matter how many processors you add; the parallel part costs $(1 - f)/p$.

2

Compute a case

With $f = 0.05$ and $p = 16$: $S = 1 / (0.05 + 0.95/16) = 1 / 0.1094 = 9.14$. Sixteen processors deliver a speedup of nine — an efficiency of 57%.

3

Take the limit

With the same $f = 0.05$ and unbounded p , the second term vanishes and $S \rightarrow 1/f = 20$. No quantity of hardware, ever, gets past twenty.

4

Read what it forbids

The bound applies to this fixed problem. It says nothing about a larger problem, and nothing about whether f itself can be reduced — which is usually where the engineering effort actually belongs.

Note the practical reading: at $f = 0.05$ the useful range of p is small, so “we ran it on 256 cores” is only impressive once the serial fraction has been reported alongside it.

Gustafson's Law and the Universal Scalability Law

Two responses to Amdahl. The first changes the premise; the second adds the mechanisms Amdahl leaves out.

GUSTAFSON, 1988

$$S(p) = p - f \cdot (p - 1)$$

Hold the running time fixed rather than the problem. If the parallel portion grows with the machine — as it does whenever people buy bigger hardware to solve bigger problems — scaled speedup is linear in p .

Same arithmetic, different premise, opposite conclusion. A clean illustration of Lecture 3's point that a theorem's content sits in its assumptions.

GUNTHER'S UNIVERSAL SCALABILITY LAW

$$C(p) = p / (1 + \alpha(p-1) + \beta p(p-1))$$

Adds two effects Amdahl omits: α is contention — queueing for a shared resource — and β is coherency, the cost of keeping p workers' data consistent. When $\beta > 0$ the throughput curve rises, peaks, and then falls.

Which is what real systems do, and what neither Amdahl nor Gustafson can express at all.

The USL's practical value: fit α and β to a handful of measured load points, and it predicts where the throughput curve turns over — the point past which adding capacity actively makes the system slower. That is a prediction, made from measurement, that can then be checked. Lecture 5's whole discipline, applied to a scaling claim.

Bottleneck Analysis with Operational Bounds

Lecture 5's operational laws again, now used to compute a ceiling before running a single experiment. No distributional assumptions are needed.

THE QUANTITIES, AND THE TWO BOUNDS

D_i = service demand at device i D = sum of all D_i $D_{\max}$ = the largest D_i
 N = number of users Z = think time

$$X(N) \leq \min(N / (D + Z), 1 / D_{\max}) \qquad R(N) \geq \max(D, N \cdot D_{\max} - Z)$$

- **The bottleneck is the device with the largest service demand.**

Not the busiest-looking one, and not the one that is easiest to optimise. $D_{\max}$ alone sets the asymptotic throughput ceiling, so improving any other device cannot raise it at all.

- **The bounds are free, and they come first.**

They need only the service demands, which a single light-load measurement gives you. You can compute the ceiling before designing the experiment — and then find out whether the system reaches it.

- **A gap between bound and measurement is diagnostic.**

If measured throughput falls well short of $1/D_{\max}$, something outside the model — contention, coherency, a software serialisation — is costing you.

WHY THIS IS METHODOLOGY

A bound computed in advance turns a measurement into a test.

Without it, a throughput number is just a number: nothing could have contradicted it, so nothing was learned by obtaining it.

With it, the measurement either confirms the bound, or reveals a mechanism the operational model does not contain. Either outcome is informative — which, by Lecture 1's standard, is what makes it an experiment at all.

Where the Model Meets the Measurement

Lecture 5 built models that predict performance. This session measures it. Comparing the two is the most informative thing you can do with either.

OUTCOME 1

They agree

The model is validated within the range you measured — and only within it. You may now use it to interpolate with confidence, and to extrapolate with stated caution.

This is the weakest of the three outcomes intellectually, and the most useful practically: you have a cheap predictor you can trust.

OUTCOME 2

They disagree

Some mechanism the model omitted is dominating. Locating it is a finding, not a failure — and it is frequently the most interesting sentence in the paper.

Contention, coherency, cache behaviour, a software serialisation. Lecture 5 called this structural error; here it has a name and a magnitude.

OUTCOME 3

The model was fitted after

If the model's parameters were tuned once the measurements were in, nothing has been validated. The model was calibrated, and calibration proves only that the fit succeeded.

Lecture 5's central distinction and Lecture 3's HARKing, arriving together in the same paragraph.

Predict first. Then measure. Then explain the gap. That sequence is what turns a table of benchmark numbers into a contribution — and it is where all five earlier sessions of this unit converge.

Matching Evaluation to the Claim

Find the claim you intend to make, then read across. The last column is the objection to pre-empt.

THE CLAIM YOU WANT TO MAKE	TECHNIQUE	WHAT TO REPORT	THE OBJECTION TO PRE-EMPT
“This is faster on workload W”	Measurement	Distribution, baseline, full configuration	Was the baseline tuned, and is W representative?
“This scales to p processors”	Measurement + scaling model	Speedup and efficiency curves, with T(1) defined	Which T(1)? And strong or weak scaling?
“This handles ten times today's load”	Analytical model or simulation	Predicted curve, plus validation at current load	Does the model hold outside the measured range?
“Component X is the bottleneck”	Profiling + operational bounds	Service demands, and the effect of removing X	Would fixing X actually move the ceiling?
“This uses less energy per request”	Measurement with power instrumentation	Joules per operation at matched throughput	Was throughput held equal, or quietly traded away?

Every row pairs a claim with the only kind of evidence that can settle it — the same discipline as Lecture 2's triangulation table, narrowed to performance.

How Performance Claims Fail

Six recurring faults. None requires dishonesty — each is an ordinary choice, made without stating it.

The arithmetic mean of rates

A summary statistic that can, and demonstrably does, name the losing system as the winner. Slide 8 was not a contrived case.

The bare mean latency

A single number summarising a skewed distribution, concealing precisely the tail that determines what users actually experience.

The unstated $T(1)$

Speedup reported against the authors' own single-threaded code rather than the best known sequential algorithm for the problem.

The unmatched configuration

Different compilers, flags, runtimes or hardware between the proposed system and its baseline — so the comparison measures the setup.

Measurement without isolation

Numbers collected on a shared or thermally throttled machine, then reported to three significant figures as though the noise were not there.

Extrapolating the curve

Measuring at twenty per cent load and asserting behaviour at ninety, where queueing makes the relationship sharply non-linear.

Run Rules: What Disciplined Benchmarking Looks Like

What SPEC and TPC actually require. Each rule exists because a specific defect was found the expensive way.

1

Full disclosure

A published configuration report: hardware, software, tuning, and every compiler flag. Results submitted without it are simply not accepted.

2

Run rules

Which optimisations are permitted, how many iterations must run, and which results may be reported — preventing tuning that helps only the benchmark.

3

Repeated measurement

A required minimum number of runs and a defined statistic over them, so that a single lucky run can never be reported on its own.

4

Audit and review

TPC results are independently audited before publication; SPEC results are reviewed by the committee before they are accepted.

5

Price and power

TPC requires a total cost of ownership figure; SPECpower requires energy measured at graduated load. Performance without cost is half a comparison.

WHAT THIS MEANS FOR A RESEARCH PAPER

You are not obliged to follow SPEC's rules in a paper — those bodies are certifying products, not publishing research. You are obliged to make a comparable disclosure, because every one of these rules encodes a defect the field discovered by being misled. Adopting them costs an appendix and buys a result that survives scrutiny.

A Checklist for Your Own Evaluation

- ✓ **Name the five dependencies.** System, workload, platform, metric, summary statistic. If any is missing from the sentence stating your result, the number cannot be reproduced or contested.
- ✓ **Justify your mean.** Arithmetic for times, harmonic for rates, geometric for normalised ratios, median for skewed data. Say which you used, and why that one.
- ✓ **Report a distribution, not a point.** Median and spread, or percentiles with the sample count — and the offered load at which they were measured.
- ✓ **Tune the baseline as hard as your own system.** And describe what you did to it. This is the single most common reason a performance result does not survive review.
- ✓ **Control what you can; report what you cannot.** Frequency scaling, memory layout, co-tenancy, NUMA placement, warm-up. Silence here reads as an uncontrolled experiment, because it is one.
- ✓ **Predict before you measure.** Compute a bound or a model prediction first, then measure, then explain the gap. That gap is usually the most interesting result you have.

Discussion

In pairs, ten minutes. Bring one answer back for class critique.

- 1 Find a performance claim in a recent paper from your area. Which of the five dependencies does it actually state, and which must you guess in order to interpret the number?
- 2 Take a speedup graph you have seen. What is $T(1)$ in it — and would using the best known sequential algorithm as the baseline change the shape of the curve, or only its height?
- 3 A service reports a p99 latency of 40 ms, and a user request fans out to fifty such services. What is the user likely to experience, and why is the answer not 40 ms?
- 4 Pick a system you can actually run. Estimate its service demands, compute the asymptotic throughput bound, then measure. Where does the measurement fall short of the bound, and what does that gap tell you?

Key Takeaways

- 1 Performance is not a property of a system. It is a property of a system, on a workload, on a platform, under a metric, summarised by a statistic — and a claim naming fewer than five of those is under-specified.
- 2 The summary statistic is a methodological choice with consequences: the arithmetic mean of rates can name the losing system as the winner, and a bare mean conceals the tail that users actually experience.
- 3 Measurement is the most persuasive technique and the easiest to get wrong. Warm-up, frequency scaling, memory layout and co-tenancy routinely produce swings larger than the speed-ups being reported.
- 4 Scaling laws are deductions inside models. Amdahl fixes the problem, Gustafson fixes the time, and the Universal Scalability Law adds contention and coherency so that throughput can peak and then fall. Each conclusion follows from its premise.
- 5 The strongest evaluations predict first and measure second, then explain the gap — which is where this unit's six sessions converge: an argument (L1), proved in a mode (L2), by an inference (L3), within a tradition (L4), through a model (L5), against a number (L6).

Further Reading

Jain, R. The Art of Computer Systems Performance Analysis. Wiley, 1991. — the standard text, carried through Lectures 2, 5 and 6.

Lilja, D. J. Measuring Computer Performance: A Practitioner's Guide. Cambridge University Press, 2000.

Lazowska, E. D. et al. Quantitative System Performance. Prentice-Hall, 1984. — operational bounds and bottleneck analysis.

Amdahl, G. M. “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities.” AFIPS, 1967.

Gustafson, J. L. “Reevaluating Amdahl's Law.” Communications of the ACM, 1988.

Gunther, N. J. Guerrilla Capacity Planning. Springer, 2007. — the Universal Scalability Law.

Mytkowicz, T., Diwan, A., Hauswirth, M. & Sweeney, P. F. “Producing Wrong Data Without Doing Anything Obviously Wrong!” ASPLOS, 2009.

Georges, A., Buytaert, D. & Eeckhout, L. “Statistically Rigorous Java Performance Evaluation.” OOPSLA, 2007.

Dean, J. & Barroso, L. A. “The Tail at Scale.” Communications of the ACM, 2013.